Oracle RRI Pipeline in ASE

Data → Candidates → Rendering → Backprojection → Metrics

Jan Duchscherer

VCML Seminar WS24/25

Oracle RRI Pipeline

Reusable compute path for dashboard, preprocessing, and training

```python
@dataclass(slots=True)
class OracleRriLabelBatch:
    sample: EfmSnippetView
    candidates: CandidateSamplingResult
    depths: CandidateDepths
    candidate_pcs: CandidatePointClouds
    rri: RriResult

```

Oracle RRI Pipeline

Inputs (per snippet)

- `EfmSnippetView`: cameras, trajectory, semi-dense points, optional GT/OBBs
- GT mesh tensors: `mesh_verts` / `mesh_faces`
- Reference pose: last trajectory pose (or chosen frame)

Stages

1. **Candidate Generation**: sample + orient + prune `PoseTW["C 12"]`
2. **Depth Rendering**: render `depths["C H W"]` (+ valid mask)
3. **Backprojection**: depth hits → `CandidatePointClouds`
4. **Oracle Scoring**: `RriResult` per candidate

Oracle RRI Pipeline

Orchestrator

- `OracleRriLabeler` composes three stage configs:
  1. `CandidateViewGeneratorConfig` (pose sampling + pruning)
  2. `CandidateDepthRendererConfig` (depth simulation from GT mesh)
  3. `OracleRRIConfig` (point↔mesh distances ⟶ RRI)
- Pipeline knob: `backprojection_stride` (pixel subsampling)

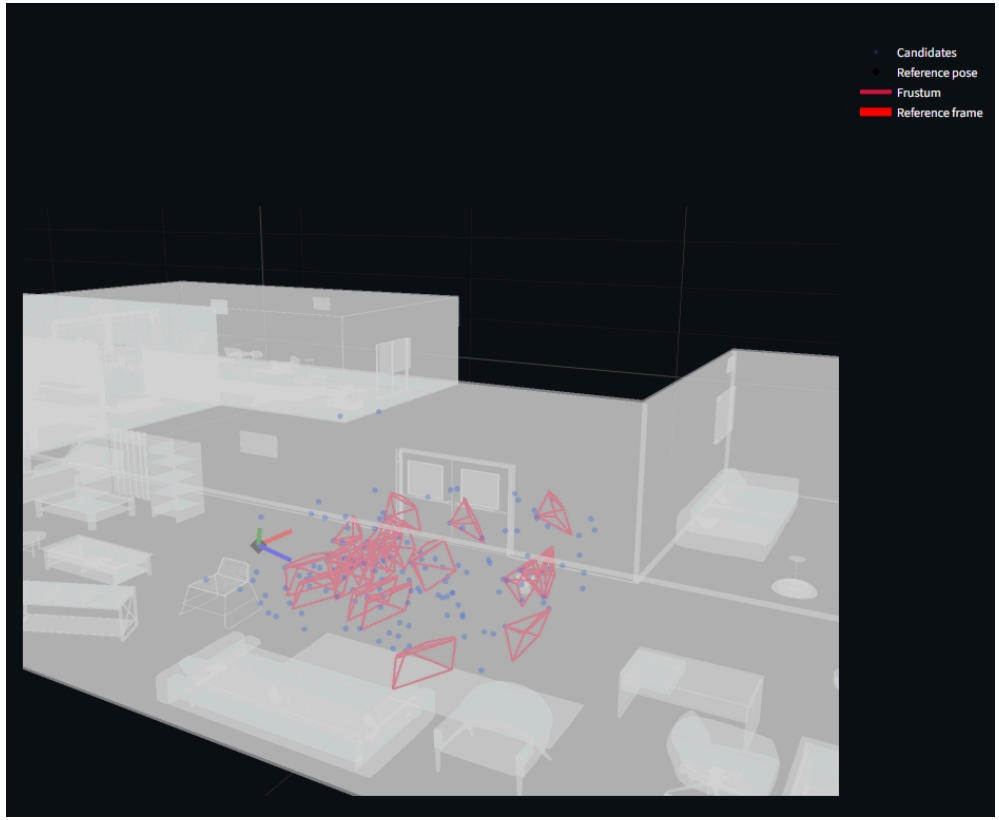
Outputs

- `OracleRriLabelBatch` contains:
  1. `candidates` (`CandidateSamplingResult`)
  2. `depths` (`CandidateDepths`)
  3. `candidate_pcs` (`CandidatePointClouds`)
  4. `rri` (`RriResult`)

> Currently engineered for plotting and debugging in the streamlit app. Results contain too many intermediate tensors for optimal batching per snippet.

Candidate Generation

Generate and prune candidate `PoseTW` around a reference pose



CandidateViewGenerator: Overview

Inputs

- Reference pose: last rig/camera pose (optionally gravity-aligned).
- Occupancy extent (AABB) from semidense bounds or mesh bounds.
- GT mesh (for clearance + collision pruning).

Outputs

- Candidate poses: `PoseTW["C 12"]` (world ← cam)
- Diagnostics: per-rule masks + optional debug stats
- Stable indexing: map back to global candidate indices for UI alignment

Candidate Centers (Theory)

Sampling distribution

Sample candidate centers on a spherical band in a reference frame:

$$r \sim \mathcal{U}(r_{\min}, r_{\max})$$

$$\varphi \sim \mathcal{U}\left(-\frac{\Delta_\varphi}{2}, \frac{\Delta_\varphi}{2}\right)$$

$$u \sim \mathcal{U}(\sin\theta_{\min}, \sin\theta_{\max})$$

$$\theta = \arcsin(u).$$

Unit direction and center:

$$\boldsymbol{s}_i = (\cos\theta_i \cos\varphi_i, \sin\theta_i, \cos\theta_i \sin\varphi_i)$$

$$\boldsymbol{p}_i = \boldsymbol{t}_{\text{ref}} + \boldsymbol{R}_{\text{ref}}(r_i \boldsymbol{s}_i)$$

$$\boldsymbol{t}_i = \boldsymbol{p}_i.$$

The orientation is assigned in the **view-direction** stage.

Practical options in code

- **Uniform**: area-uniform band sampling (`UNIFORM_SPHERE`).
- **Forward-biased**: PowerSpherical / von-Mises–Fisher concentration (`kappa`) around forward axis.
- **Gravity alignment**: remove pitch/roll in the sampling frame (`align_to_gravity=True`).

Intuition

- Explore around the last pose while respecting the rig's reachable neighborhood.
- Limit elevation to remain "human-like" (no ceiling cams).
- Keep a controllable azimuth spread for forward exploration vs. full sweep.

Candidate Orientations (Theory)

Goal

Build a camera rotation `R_world_cam` so each candidate **looks at** a target (often the last pose), while keeping roll stable and respecting the Aria LUF convention:

- +x: left, +y: up, +z: forward

Look-at frame

For camera center `p` and target $\boldsymbol{p}_c$:

$$\boldsymbol{z} = \frac{\boldsymbol{p}_c - \boldsymbol{p}}{\|\boldsymbol{p}_c - \boldsymbol{p}\|}$$

$$\boldsymbol{y} = \boldsymbol{u}_{\text{world}}$$

$$\boldsymbol{x} = \frac{\boldsymbol{y} \times \boldsymbol{z}}{\|\boldsymbol{y} \times \boldsymbol{z}\|}.$$

Then re-orthogonalize: $\boldsymbol{y} = \boldsymbol{z} \times \boldsymbol{x}$.

What defines the base view?

- `view_direction_mode` picks a deterministic base orientation:
  1. `radial_away` / `radial_towards`: look along (candidate ↔ reference) with roll-free world-up.
  2. `forward_rig`: reuse the rig rotation.
  3. `target_point`: look-at a fixed point.
- This yields $\boldsymbol{R}_{\text{base}}$.

> **Roll-free?** One less DOF to worry about!

View-Direction Randomization

View Direction Randomization

- Jitter is then applied as a **right-multiplicative** delta:

$$\boldsymbol{R}_{\text{final}} = \boldsymbol{R}_{\text{base}} \cdot \boldsymbol{R}_{\text{delta}}.$$

> This makes jitter "relative to the candidate's current view direction": `0°` means "keep the base view".

Caps vs sampler (priority order)

bounded uniform jitter around the base direction via `view_max_azimuth_deg` / `view_max_elevation_deg`:

Sample angles (caps in radians):

$$\alpha \sim \mathcal{U}(-\alpha_{\max}, +\alpha_{\max})$$
$$\beta \sim \mathcal{U}(-\beta_{\max}, +\beta_{\max}).$$

Convert to a camera-frame forward direction (LUF):

$$\boldsymbol{z}_{\text{delta}} = (\cos\beta \sin\alpha, \sin\beta, \cos\beta \cos\alpha).$$

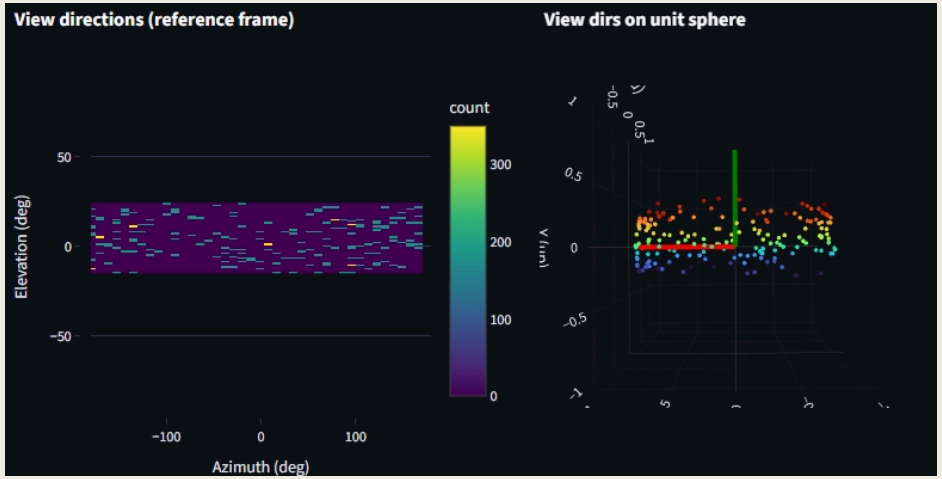
Build a right-multiplicative delta rotation and compose:

$$\boldsymbol{R}_{\text{delta}} = \boldsymbol{R}_{y(\alpha)} \boldsymbol{R}_{x(\beta)}$$
$$\boldsymbol{R}_{\text{final}} = \boldsymbol{R}_{\text{base}} \cdot \boldsymbol{R}_{\text{delta}}.$$

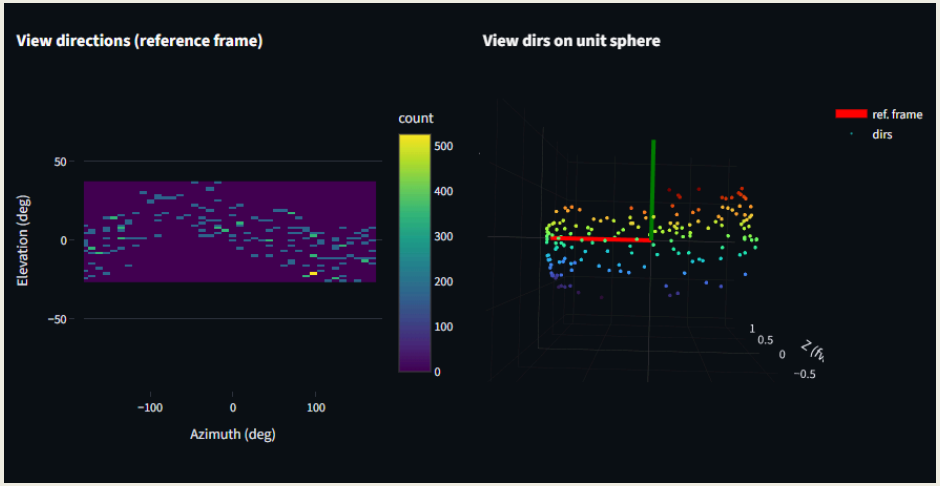
Optional roll jitter:

Pose-aligned vs Gravity-aligned Sampling

Pose-aligned (`align_to_gravity=false`)

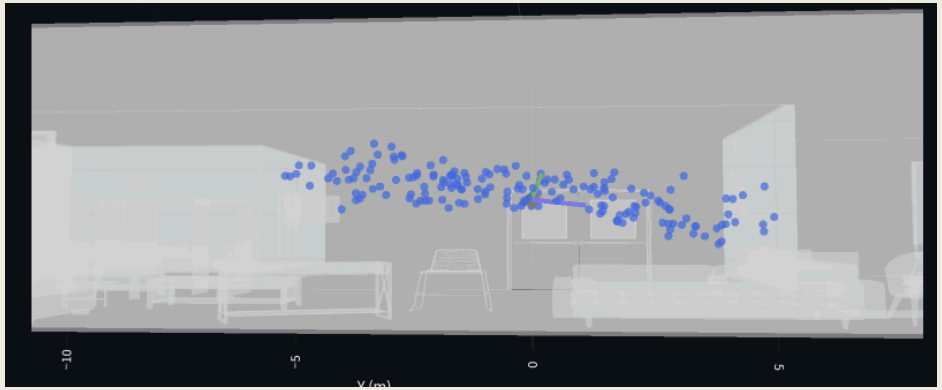


Gravity-aligned (`align_to_gravity=true`)

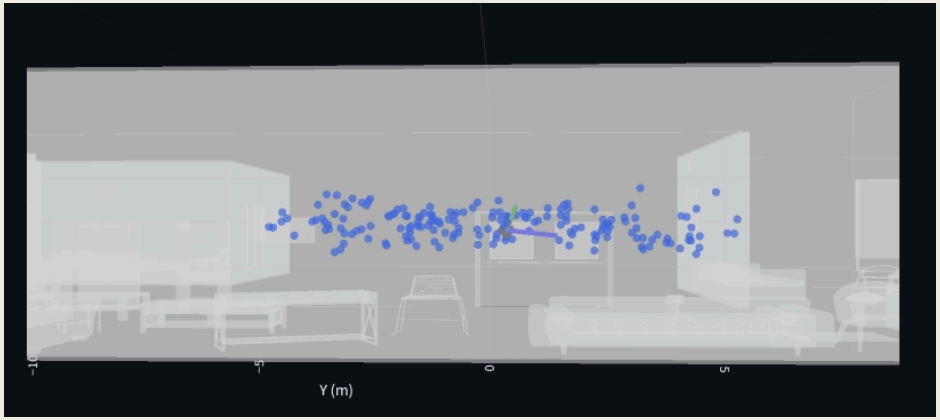


Pose-aligned vs Gravity-aligned Sampling

Pose-aligned (`align_to_gravity=false`)

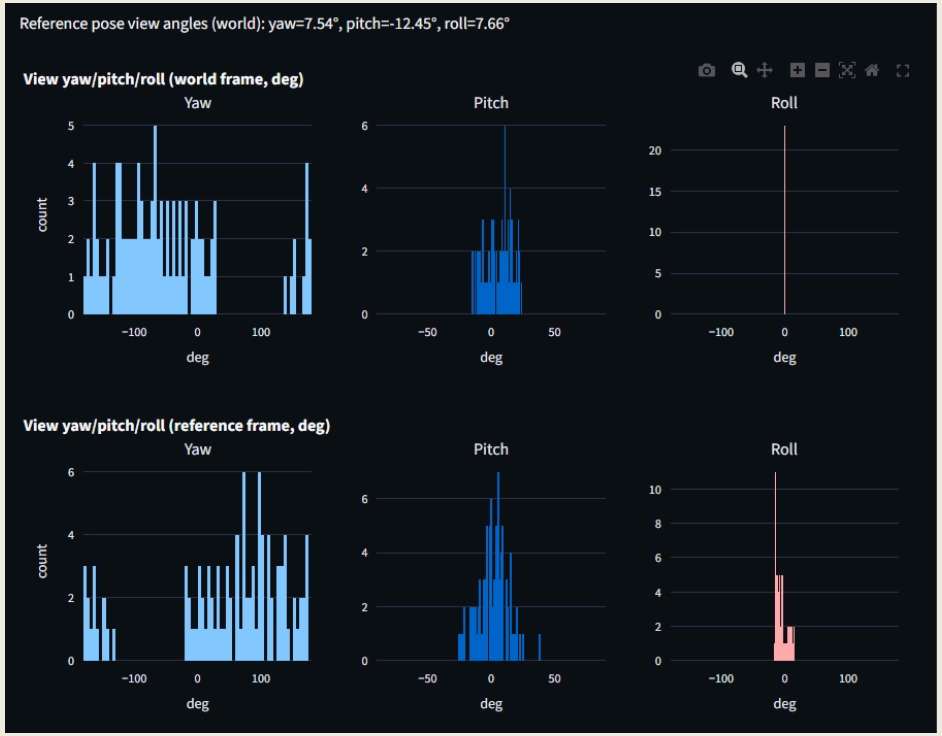


Gravity-aligned (`align_to_gravity=true`)

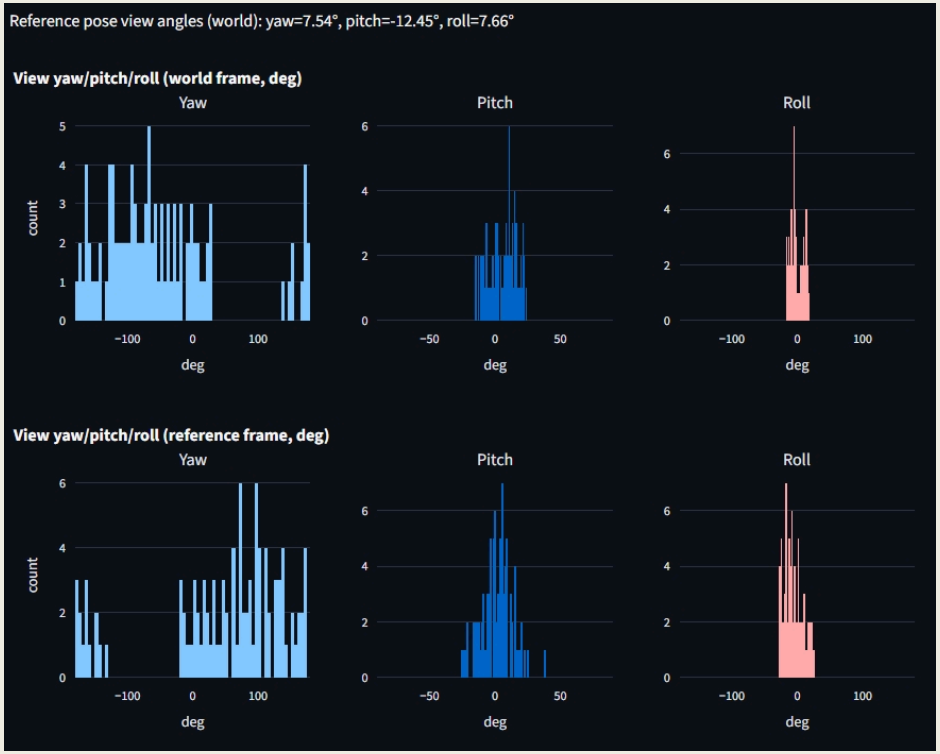


Roll Jitter: What Changes?

No roll jitter (`view_roll_jitter_deg=0`)



With roll jitter (`view_roll_jitter_deg≈17°`)

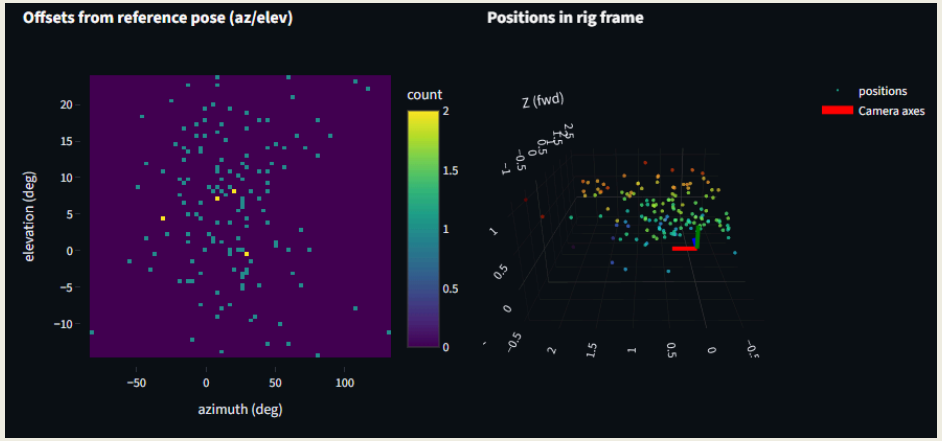


Roll Jitter: What Changes?

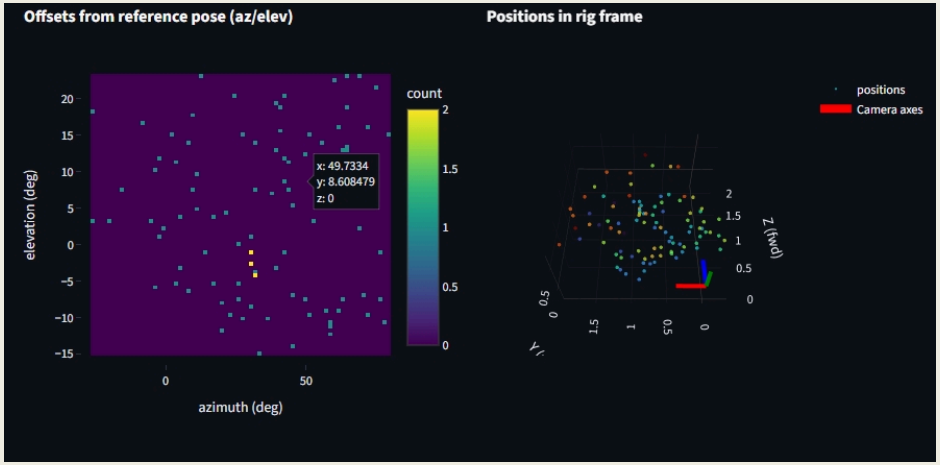
> Roll jitter changes the **twist** about the forward axis (roll histogram spreads), while azimuth/elevation of view directions stays governed by `view_direction_mode` and the az/el caps.

Forward Bias: Candidate Center Distributions

Forward-biased (PowerSpherical, `kappa=4`, `r=0.6..2.9`)



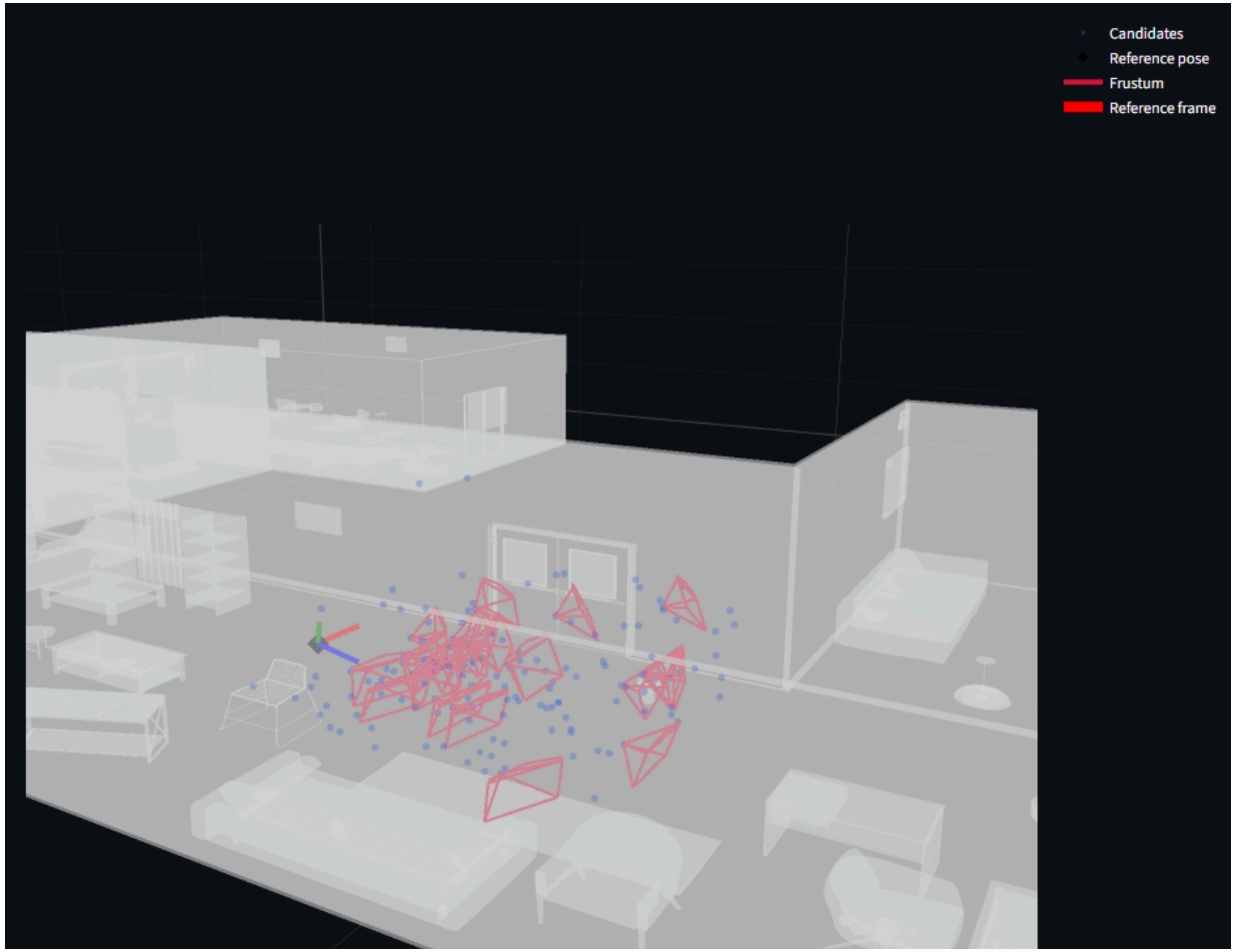
Narrower azimuth + stronger pruning (`Δaz≈165°`, `min_dist≈1m`)



Forward Bias: Candidate Center Distributions

> Forward bias (PowerSpherical, `kappa>0`) concentrates candidate centers near the current forward direction. This matches realistic motion (users rarely turn 180° for the next view), reduces wasted renders on implausible “behind-the-back” candidates, and avoids over-emphasizing views that may score high oracle RRI but are unlikely to be executed.

Forward-biased Candidates: Frusta in Scene



Reading the plot

- Blue points: candidate centers after pruning.
- Red wireframes: rendered camera frusta (subset).
- Axes: reference frame at the sampling origin.
- In `radial_away`, the frusta optical axes align with the center direction (up to optional view jitter).

Rule: Free-Space AABB

Criterion

Constrain candidate centers to an occupancy extent:

$$x_{\min} \leq x_i \leq x_{\max},$$
$$y_{\min} \leq y_i \leq y_{\max},$$
$$z_{\min} \leq z_i \leq z_{\max}.$$

Intuition

- Prevent candidates from leaking outside the local room/scene bounds.
- Bounds come from semidense metadata (`volume_min/max`) or mesh crop bounds.
- Cheap to evaluate (first pruning stage).

Rule: Min Distance to Mesh

Criterion

For candidate center $\boldsymbol{c}_i$ and mesh $\mathcal{M}$:

$$d_i = \min_{\boldsymbol{x} \in \mathcal{M}} \|\boldsymbol{c}_i - \boldsymbol{x}\|_2.$$

Keep iff $d_i > d_{\min}$.

Intuition

- Reject viewpoints that start inside/too close to GT geometry.
- Prevents unstable renders (near-plane clipping explosions).
- Implemented via PyTorch3D point↔mesh distance or trimesh proximity queries.

Rule: Path Collision

Criterion

Straight-line segment from reference pose $\boldsymbol{o}$ to candidate center $\boldsymbol{p}_i$:

$$\boldsymbol{r}_i(t) = \boldsymbol{o} + t\hat{\boldsymbol{d}}_i, t \in [0, \|\boldsymbol{p}_i - \boldsymbol{o}\|].$$

Reject if the segment intersects the mesh (optionally with clearance).

Intuition

- Enforce collision-free translation in a simple
- Backends:
  1. PyTorch3D distance sampling along the segment (GPU)
  2. Trimesh / PyEmbree ray intersector (CPU)

Pose Generation Diagram

Figure 1: oracle_rri.pose_generation subpackage

Depth Rendering

Render candidate depth maps from the GT mesh

Implemented in `oracle_rri/oracle_rri/rendering/`.

CandidateDepthRenderer: Responsibilities

Candidate selection

- Render only a subset for performance:
  1. oversample `oversample_factor`
  2. cap to `max_candidates_final`
- Keep stable indices via `candidate_indices` (maps into the full candidate list).

Valid depth mask

We treat a pixel as valid iff:

- `pix_to_face >= 0`
- `z > z_near`
- `z < z_far`

Misses often appear as `pix_to_face=-1` (and may show `z=-1` in histograms).

> **Practical tip**: Always plot depth **with** the valid mask applied; otherwise invalid pixels dominate histograms.

CandidateDepthRenderer: Responsibilities

Outputs

- `depths["C H W"]` in metres (metric z-depth)
- `depths_valid_mask["C H W"]` (hit + near/far clipping)
- `poses` (world ← cam) and `p3d_cameras` for later backprojection

PyTorch3D Rasterization (Conceptual)

Camera model

We use PyTorch3D `PerspectiveCameras` in **screen space** (`in_ndc=false`):

$$u = f_x \frac{X}{Z} + c_x, v = f_y \frac{Y}{Z} + c_y.$$

The renderer internally converts screen coordinates to NDC using $s = \min(H, W)$ (important for non-square images).

Coordinate conventions

- PyTorch3D NDC: +X left, +Y up, right-handed.
- We keep all core geometry in the physical Aria LUF world frame.
- Extrinsics must be consistent with `PoseTW` conventions when building `PerspectiveCameras(R,T)`.

PyTorch3D Rasterization (Conceptual)

Rasterization output

`MeshRasterizer` returns fragments:

- `pix_to_face`: face index per pixel (hit/miss)
- `zbuf`: per-pixel z-depth (metres when configured consistently)
- optional barycentric coordinates (not used for oracle depth)

Why rasterization?

- GPU batch rendering for many candidates
- Deterministic depth (z-buffer)
- Works directly with mesh tensors (`verts`, `faces`)

Backprojection

Depth hits → candidate point clouds (world frame)

Implemented in `unproject.py` and `candidate_pointclouds.py`.

Depth → 3D Points (Math)

Pixel → NDC mapping (PyTorch3D)

For pixel indices `(x,y)` and pixel centers:

$$u = x + 0.5,$$
$$v = y + 0.5,$$
$$s = \min(H, W).$$

Convert to NDC:

$$x_{\text{ndc}} = -\left(u - \frac{W}{2}\right)\left(\frac{2}{s}\right), y_{\text{ndc}} = -\left(v - \frac{H}{2}\right)\left(\frac{2}{s}\right).$$

Form `xy_depth = (x_ndc, y_ndc, z)`.

Unprojection

PyTorch3D provides:

- `unproject_points(xy_depth, world_coordinates=true, from_ndc=true)`
- Output: `p_world` for each valid pixel.

Why NDC here?

Using NDC consistently matches the internal camera transform used by the rasterizer. This avoids subtle “frustum ≠ points” bugs for non-square images.

Vectorized Backprojection (Implementation)

Batch strategy

- Rendered depths: `depths["B H W"]`, mask: `mask_valid["B H W"]`.
- Sample strided pixel grid (stride = `backprojection_stride`).
- Build batched `xy_depth["B P 3"]` and call `cameras.unproject_points(...)` once.

Fusion for scoring

- Collapse semi-dense SLAM points:
  1. `semidense_points["K 3"]` (NaNs removed, time collapsed)
- Compute combined bounds:
  1. `occupancy_bounds[6]` = union of snippet AABB + candidate PCs + semidense

> Stride is a key compute knob: smaller stride = denser candidate PCs, but more GPU work in both unprojection and point↔mesh distances.

Vectorized Backprojection (Implementation)

Packing valid hits

- Compute `mask["B P"]` for valid pixels.
- Compact points per batch into:
  1. `points["B Pmax 3"]` (padded with NaNs)
  2. `lengths["B"]` (valid point counts)

Oracle RRI Metric

Point↔mesh distances (accuracy + completeness) → normalized improvement

Implemented in `oracle_rri/oracle_rri/rri_metrics/`.

RRI Definition

Relative Reconstruction Improvement

For candidate view $\boldsymbol{q}$:

$$\mathrm{RRI}(\boldsymbol{q}) = \frac{d(\mathcal{P}_t, \mathcal{M}_{\mathrm{GT}}) - d\left(\mathcal{P}_{\{t \cup \boldsymbol{q}\}}, \mathcal{M}_{\mathrm{GT}}\right)}{d(\mathcal{P}_t, \mathcal{M}_{\mathrm{GT}})}.$$

- $\mathcal{P}_t$: semi-dense SLAM reconstruction up to time t
- $\mathcal{P}_{\{t \cup \boldsymbol{q}\}}$: merged reconstruction after backprojecting candidate depth hits
- $d(.,.)$: **bidirectional** point↔mesh distance

> The ratio cancels units and makes scores comparable across scenes and scales.

Point↔Mesh Distance: Accuracy + Completeness

Accuracy (P → M)

Average distance from reconstruction points to the GT surface:

$$d_{\{\mathcal{P}\to\mathcal{M}\}} = \frac{1}{|\mathcal{P}|} \sum_{\{\boldsymbol{p}\in\mathcal{P}\}} \min_{\{\boldsymbol{x}\in\mathcal{M}\}} \|\boldsymbol{p}-\boldsymbol{x}\|_2^2.$$

- Detects **over-reconstruction** (spurious geometry).
- Implemented via PyTorch3D `point_face_distance` (point→triangle).

Completeness (M → P)

Average distance from GT surface elements to the reconstruction:

$$d_{\{\mathcal{M}\to\mathcal{P}\}} \approx \frac{1}{|\mathcal{F}|} \sum_{\{\boldsymbol{f}\in\mathcal{F}\}} \min_{\{\boldsymbol{p}\in\mathcal{P}\}} d(\boldsymbol{f},\boldsymbol{p})^2.$$

- Detects **under-reconstruction** (missing geometry).
- Implemented via PyTorch3D `face_point_distance` (triangle→point).

Bidirectional

$$d_{\{\mathcal{P}\leftrightarrow\mathcal{M}\}} = d_{\{\mathcal{P}\to\mathcal{M}\}} + d_{\{\mathcal{M}\to\mathcal{P}\}}.$$

chamfer_point_mesh_batched: Fully Vectorized Breakdown

Inputs

- Candidate batch: `points["C P 3"]` (padded) + `lengths["C"]`
- Mesh: `gt_verts["V 3"]`, `gt_faces["F 3"]`
- Output: `DistanceBreakdown` with `accuracy`, `completeness`, `bidirectional` (per candidate)

2) Compute distances

PyTorch3D returns packed distances:

- point→face: `(P_tot,)` (each point to nearest triangle)
- face→point: `(F_tot,)` (each face to nearest point)

3) Reduce per candidate

```
accuracy[c]      = sum(point_to_face[p in c]) / |P|
completeness[c]  = sum(face_to_point[f in c]) / |F|
bidirectional[c] = accuracy[c] + completeness[c]
```

chamfer_point_mesh_batched: Fully Vectorized Breakdown

HM

1) Pack points

Turn padded `(C,P,3)` into a packed `(P_tot,3)` representation + index maps:

```
mask = arange(P)[None, :] < lengths[:, None]    # (C,P) bool
points_packed = points[mask]  # (P_tot,3)

points_first_idx = cumsum(lengths) - lengths    # (C,)
point_to_cloud_idx = repeat_interleave(arange(C), lengths)  # (P_tot,)
```

OracleRRI.score: Candidate Batch Computation

Before / After point sets

- $\mathcal{P}_t$: collapsed semi-dense SLAM points
- For each candidate view $\boldsymbol{q}$:
  1. $\mathcal{P}_q$: backprojected depth-hit points
  2. $\mathcal{P}_{\{t \cup \boldsymbol{q}\}} = \mathcal{P}_t \cup \mathcal{P}_q$

Notes & knobs

- **Density matching**: optional voxel downsampling (`voxel_size_m`) to equalize $\mathcal{P}_t$ vs. $\mathcal{P}_{\{t \cup \boldsymbol{q}\}}$ densities.
- **Cropping**: mesh can be cropped to `occupancy_bounds` to avoid unrelated geometry.
- **Numerical safety**: clamp denominator to avoid divide-by-zero.

> The oracle is only used to generate training labels; at test time, the learned predictor replaces this expensive pipeline.

OracleRRI.score: Candidate Batch Computation

Implementation sketch

1. `dist_before = chamfer_point_mesh(P_t, M_gt)`
2. Tile `P_t` across candidates and concatenate: `P_tq = cat([P_t, P_q])`
3. `dist_after = chamfer_point_mesh_batched(P_tq, lengths_tq, M_gt)`
4. `rri = (dist_before - dist_after) / dist_before`

VIN: View Introspection Network

Predict per-candidate RRI from frozen EVL features + a shell-aware pose descriptor

Goal: replace the expensive oracle pipeline with a lightweight predictor trained on oracle labels (VIN-NBV style ordinal regression [Fra+25]).

VIN: I/O + Shapes

Inputs (symbols)

- Snippet batch size: B.
- Candidate count per snippet: N_c.
- EVL input: raw EFM snippet dict `efm: dict[str, Any]`.
- Candidate poses: `PoseTW["B N_c 12"]` (world←camera).
- Reference pose: `PoseTW["B 12"]` (world←rig).
- Training-aligned candidates: `T_camera_rig` (camera←rig) from `OracleRriLabelBatch.depths.camera`.

Decision rule

Select the predicted best next view:

$$\hat{\boldsymbol{q}} = \mathrm{argmax}_i \hat{s}_i \approx \mathrm{argmax}_i \ \mathrm{RRI}(\boldsymbol{q}_i).$$

> EVL is frozen. Only pose-encoder + scorer head are trained.

VIN: I/O + Shapes

Outputs

- CORAL logits: `logits["B N_c (K-1)"]` for K ordinal classes.
- Candidate score: $\hat{s} \in [0, 1]$ from expected ordinal value.
- Validity mask: `candidate_valid["B N_c"]` (inside EVL voxel bounds).

EVL Feature Contract (frozen)

Why the 3D neck features?

- Stable, information-rich voxel representation.
- Avoids bottlenecking VIN through task-specific head predictions.
- Efficient to query for many candidates (pool + sampling).

Implementation location

- Single adapter: `oracle_rri/oracle_rri/vin/backbone_evl.py`
- Returns a minimal “feature contract” so VIN stays insulated from upstream EVL input/output changes.

> Keep all EFM key expectations inside the EVL adapter; patch one place if schema changes.

EVL Feature Contract (frozen)

Feature tensors we use

- `neck/occ_feat["B Cocc D H W"]` (geometry/context)
- `neck/obb_feat["B Cobb D H W"]` (optional semantics/context)
- `voxel/T_world_voxel: PoseTW["B 12"]` (world←voxel)
- `voxel_extent["B 6"]` (voxel-frame AABB in metres)

VIN: Descriptor + Encodings (shell_sh)

Descriptor (rig frame) + shapes

From the training-aligned pose `T_camera_rig` (camera←rig) we invert:

$$\boldsymbol{T}_{\text{rig}\leftarrow\text{cam}} = \left(\boldsymbol{T}_{\text{cam}\leftarrow\text{rig}}\right)^{-1}.$$

Define:

$$r = \|\boldsymbol{t}\|,$$

$$\boldsymbol{u} = \frac{\boldsymbol{t}}{r+\varepsilon},$$

$$\boldsymbol{f} = \boldsymbol{R}\boldsymbol{z}_{\text{cam}},$$

$$s = <\boldsymbol{f}, -\boldsymbol{u}> .$$

Shapes (symbols):

- $\boldsymbol{t}$: `(B, N_c, 3)`, r: `(B, N_c, 1)`
- $\boldsymbol{u}$: `(B, N_c, 3)`, $\boldsymbol{f}$: `(B, N_c, 3)`, s: `(B, N_c, 1)`

If candidates use `view_direction_mode=radial_away`, then $\boldsymbol{f}$ often aligns with $\boldsymbol{u}$ (so $s \approx -1$).

VIN: Descriptor + Encodings (shell_sh)

Which encoding for which feature?

Encodings (default):

- SH(L): $\boldsymbol{u}$ and $\boldsymbol{f}$ (directions on $\mathbb{S}^2$).
- 1D Fourier: r (scalar radius in metres).
- Scalar MLP: s (and optional extra scalars).

Pose embedding:

$$\boldsymbol{E}_{\text{pose}} = [\boldsymbol{E}_u, \boldsymbol{E}_f, \boldsymbol{E}_r, \boldsymbol{E}_s] \in \mathbb{R}^{B, N_c, d_{\text{pose}}}.$$

Note: We do not use $\log(\mathrm{SO}(3))$ / `so3log` here.

VIN Head: Feature Aggregation + Scoring

Scene features from EVL

For each snippet:

- Run frozen EVL once $\longrightarrow$ `occ_feat`, `obb_feat`, `T_world_voxel`, `voxel_extent`.
- Global context: mean + max pool over (D, H, W) (occ and optionally obb).

Planned v0.2 upgrade: frustum query

Center sampling is a weak view descriptor. Next step:

- Sample K points along a small ray set in front of the camera
- Transform to voxel coordinates and sample features at those points
- Pool (mean+max) with strict point-level validity masking

> This approximates "what the candidate would see" without rendering, and stays compatible with EVL's voxel feature interface.

VIN Head: Feature Aggregation + Scoring

Candidate-conditioned query (current baseline)

- Local query: sample voxel features at the candidate camera center.
- Define `candidate_valid` if the camera center is inside voxel extent.
- Concatenate:
  1. `E_pose` (SH + radius FF + scalars)
  2. `E_global` (pooled voxel context)
  3. `E_local` (sampled voxel features)
- MLP $\longrightarrow$ `CoralLayer` $\longrightarrow$ logits.

VIN: Shell Descriptor (concept + statistics)

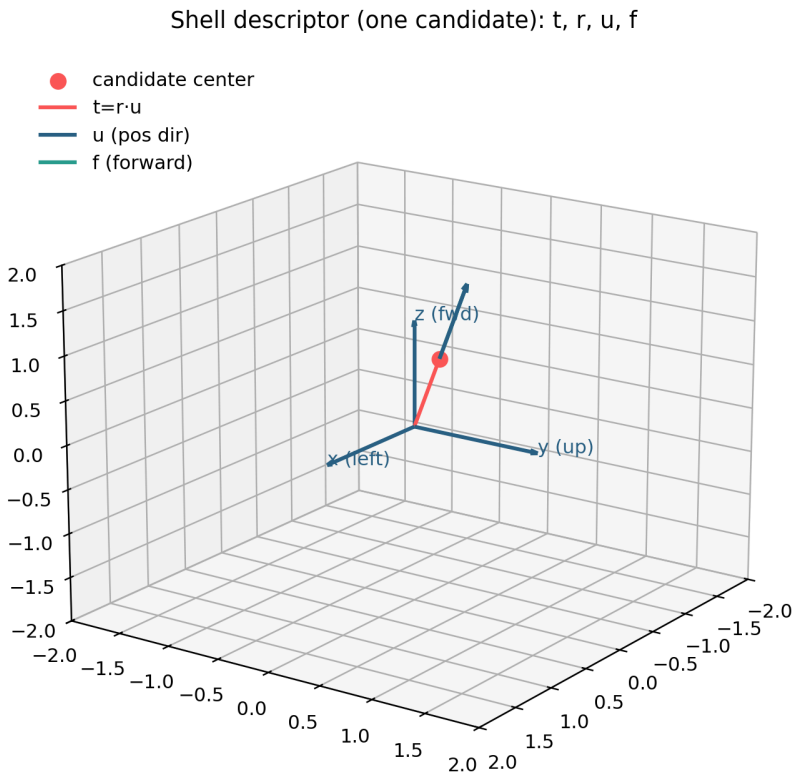


Figure 2: Conceptual descriptor for one candidate: $\boldsymbol{t}$, $r = \|\boldsymbol{t}\|$, $\boldsymbol{u}$, $\boldsymbol{f}$.

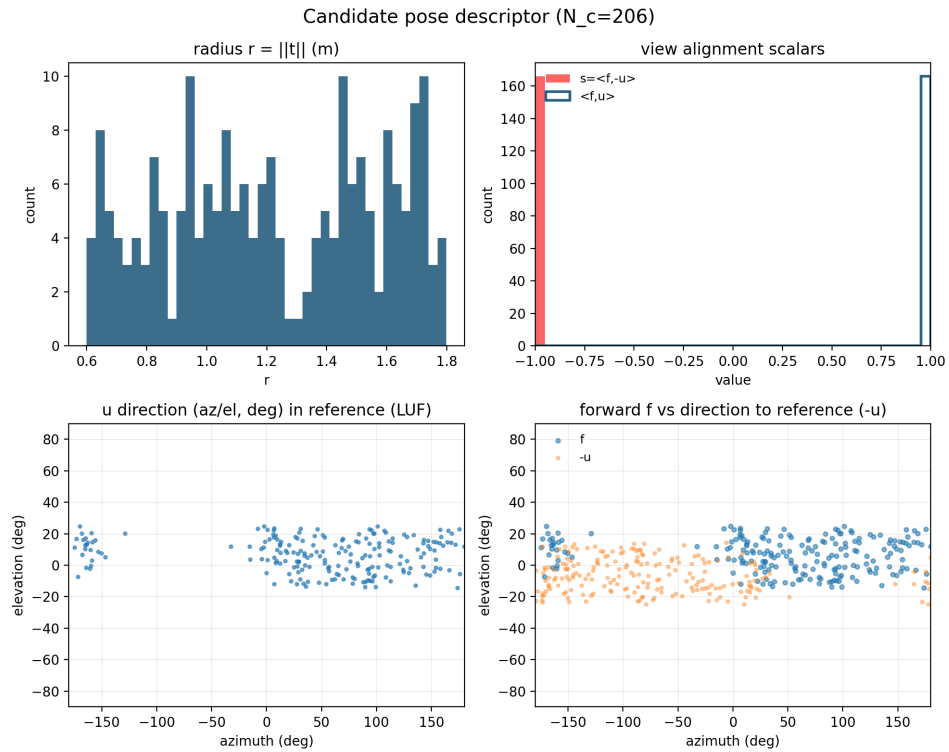


Figure 3: Empirical stats (one snippet): r, $s = < \boldsymbol{f}, -\boldsymbol{u} >$, and az/el of $\boldsymbol{u}$ vs $\boldsymbol{f}$.

VIN: Encoding Visualizations (SH + radius Fourier)

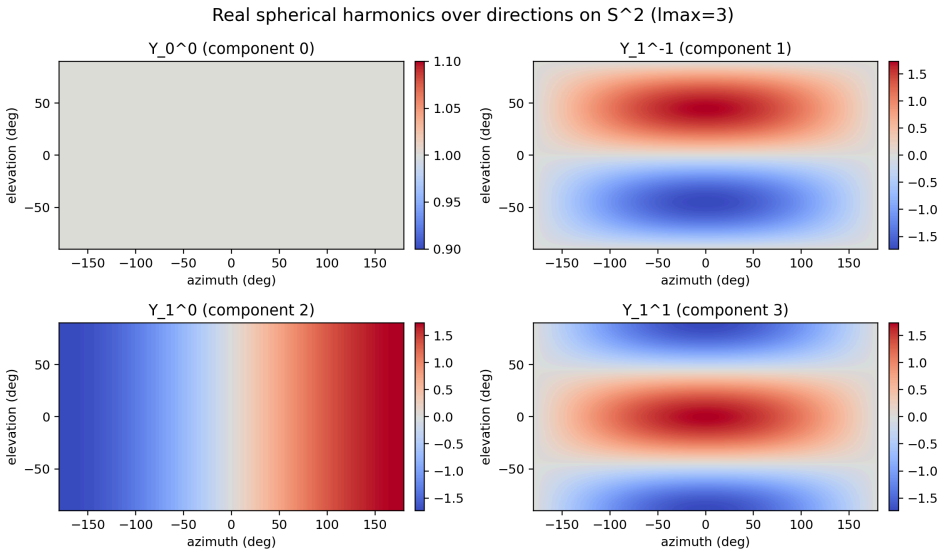


Figure 4: Real spherical harmonics components over directions on $\mathbb{S}^2$ ($L = 1$ shown).

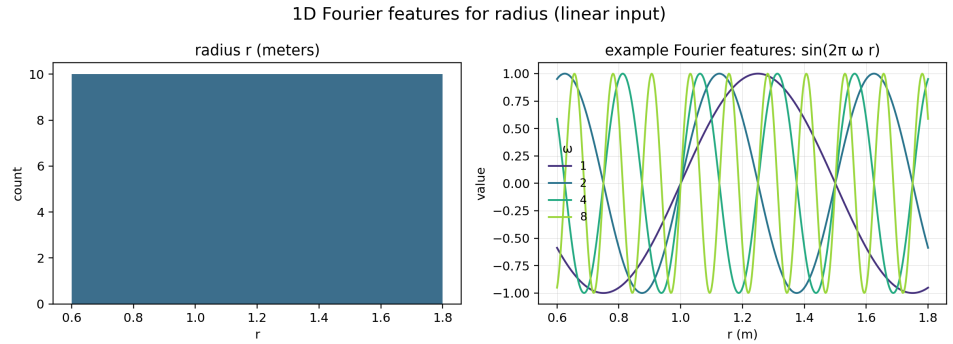


Figure 5: 1D Fourier features for radius (linear input): example $\sin(2\pi\omega r)$ curves.

VIN: EVL Neck Feature Diagnostics (frozen backbone)

HM

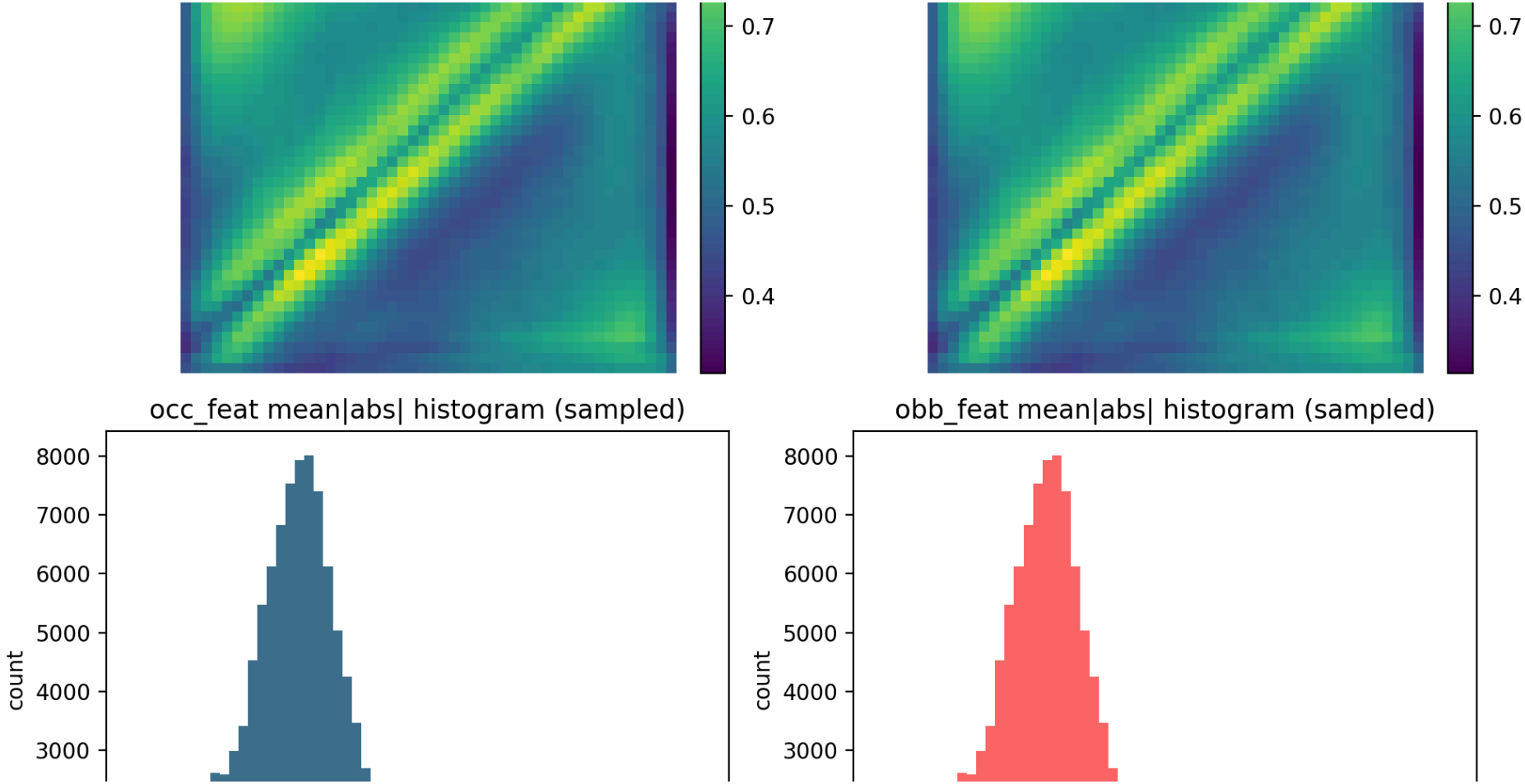


Figure 6: Mean absolute feature magnitude for `neck/occ_feat` and `neck/obb_feat` (mid-slice + histogram).

VIN: Model Summary (rich)

7	vin.pose_encoder_sh._proj_u	Sequential	1.6 K
8	vin.pose_encoder_sh._proj_u.0	Linear	544
9	vin.pose_encoder_sh._proj_u.1	GELU	0
10	vin.pose_encoder_sh._proj_u.2	Linear	1.1 K
11	vin.pose_encoder_sh._proj_f	Sequential	1.6 K
12	vin.pose_encoder_sh._proj_f.0	Linear	544
13	vin.pose_encoder_sh._proj_f.1	GELU	0
14	vin.pose_encoder_sh._proj_f.2	Linear	1.1 K
15	vin.pose_encoder_sh._radius_ff	FourierFeatures	8
16	vin.pose_encoder_sh._proj_r	Sequential	1.6 K
17	vin.pose_encoder_sh._proj_r.0	Linear	576
18	vin.pose_encoder_sh._proj_r.1	GELU	0
19	vin.pose_encoder_sh._proj_r.2	Linear	1.1 K
20	vin.pose_encoder_sh._scalar_mlp	Sequential	2.2 K
21	vin.pose_encoder_sh._scalar_mlp.0	Linear	128
22	vin.pose_encoder_sh._scalar_mlp.1	GELU	0

Figure 7: Model overview (trainable modules + parameter counts) generated with `oracle_rri.utils.rich_summary`.

Ordinal Binning: Thresholds + Example

Fit once: map scalar RRI → ordinal label

We fit a single global set of edges $e \in \mathbb{R}^{\{K-1\}}$ in clipped z-score space:

$$z = \frac{\mathrm{rri} - \mu_s}{\sigma_s + \varepsilon},$$

$$z_{\mathrm{clip}} = \tanh\left(\frac{z}{\tau}\right),$$

$$y = \sum_{j=1}^{K-1} \mathbb{1}_{z_{\mathrm{clip}} > e_j}.$$

- Choose e by quantiles of z_{clip} (bins balanced).
- Thresholds are global (not per-candidate-set) so scores remain comparable across snippets.

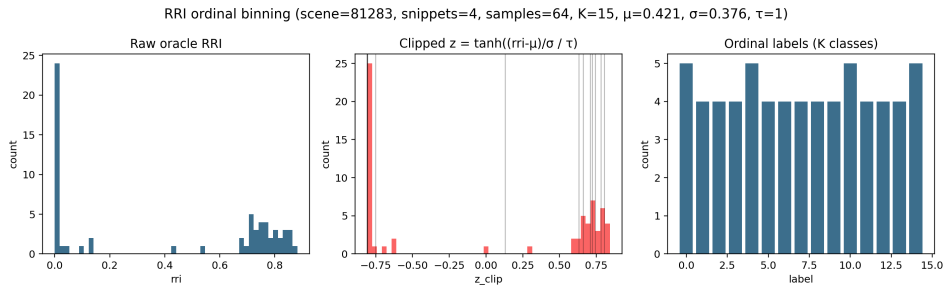


Figure 8: Example: raw RRI, clipped z distribution with quantile edges, and resulting label histogram.

Ordinal Binning: Thresholds + Example

Where it lives

- `oracle_rri/oracle_rri/vin/rri_binning.py`
- saved once (JSON) and reused during training/inference

CORAL Loss + Scoring

CORAL outputs

For K ordinal classes, predict $(K-1)$ threshold logits:

- l_k parameterizes $P(y > k)$ via $\sigma(l_k)$.
- Level targets:

$$t_k = \mathbb{1}_{y>k},$$
$$k = 0, \ldots, K-2.$$

Reference implementation

We use the MIT-licensed upstream implementation: coral-pytorch.

Masking (must-do)

Compute loss only for valid candidates:

- inside voxel grid: `candidate_valid`
- finite labels: `isfinite(rri)`
- optionally: require sufficient valid depth hits (future refinement)

CORAL Loss + Scoring

Loss + expected score

CORAL loss is a sum of binary cross-entropies:

$$\mathcal{L}_{\text{CORAL}} = \sum_{\{k=0\}}^{\{K-2\}} \text{BCE}(\sigma(l_k), t_k).$$

Score for selection uses the expected ordinal value:

$$\hat{y} = \sum_{\{k=0\}}^{\{K-2\}} \sigma(l_k),$$

$$\hat{s} = \frac{\hat{y}}{K-1}.$$

Training Step: End-to-end labeler → VIN → loss

Data path (one optimizer step)

1. Sample `EfmSnippetView` from `AseEfmDataset`.
2. Run `OracleRriLabeler.run(sample)`:
   1. candidates → depth renders → backprojection → oracle RRI
3. Convert RRI to ordinal labels via fitted binner.
4. VIN forward (EVL frozen, head trainable) → CORAL logits.
5. Backprop CORAL loss over valid candidates only.

Minimal script (smoke test)

```
cd oracle_rri
uv run python scripts/train_vin.py \
  --fit-snippets 2 --max-steps 10 --max-candidates 8 --device auto
```

Alignment detail

Train on the exact rendered subset:

- world poses: `label_batch.depths.poses`
- pose descriptor: `label_batch.depths.camera.T_camera_rig`
- reference: `label_batch.depths.reference_pose`

Training Step: End-to-end labeler → VIN → loss

> Online oracle labels are expensive. The minimal script is a correctness/smoke test; real training should cache labels (depths/PCs or RRIs).

Open Questions (VIN design choices)

Backbone features

- Use only `occ_feat` or also `obb_feat` (semantics) for RRI prediction?
- Compress voxel channels (1×1×1 conv) before pooling/sampling?

Candidate-conditioned query

- Center sampling vs. frustum point sampling (rays/depths) vs. tiny CA over points?

Pose encoding

- SH degree L (2 vs 3) and normalization choice; include camera **up** direction?
- Which scalar terms help most: $< \boldsymbol{f}, -\boldsymbol{u} >$, height, gravity alignment, etc.?

Open Questions (VIN design choices)

Ordinal setup + training

- K bins (15 default): does more resolution help or just add label noise?
- Metrics to optimize/report: Spearman rank corr, top-k recall, calibration of predicted scores.

Engineering Progress

Resolved issues and shipped tooling

Condensed from `docs/contents/todos.qmd` (resolved items + “Previously observed issues”).

Resolved Issues (High Impact)

App + pipeline refactor

- Separated UI (Streamlit panels) from compute (`OracleRriLabeler`) for reuse in training/CLI.
- Replaced untyped Streamlit caches with a typed `AppState` + explicit stage caches.

Correctness + reproducibility

- Stabilized candidate indexing: rendered subset tracked via `CandidateDepths.candidate_indices`.
- Candidate PCs align with the GT mesh (PoseTW↔PyTorch3D extrinsics fixed).
- Candidate PCs match the rendered frusta (pixel-center→NDC backprojection fixed).
- Depth histograms no longer mislead (miss pixels masked via `depths_valid_mask`).

Rendering & Backprojection: Issues and Fixes

Observed Issues

- Backprojected points “all lie in front” of the reference pose.
- Depth maps invalid for candidates that look away from the reference.
- Frusta visualization didn’t match the backprojected points.
- “Look-through-walls” artefacts (points behind walls).

Root causes + fixes

- **PoseTW ↔ PyTorch3D extrinsics mismatch**:
  1. Fix camera extrinsics so rendering/ unprojection uses the same world→camera mapping as `PoseTW`.
- **Wrong unprojection space**:
  1. Convert pixel centers `(x+0.5, y+0.5)` to NDC (min-side scaling) and unproject with `from_ndc=true`.
- **Histogram semantics**:
  1. Mask miss pixels (`depths_valid_mask`) when computing hit ratios / histograms / backprojection.
- **Rig-basis vs display twist**:
  1. `rotate_yaw_cw90` is a fixed 90° twist
  3. Apply frustum twist at rig-basis immediate

Bibliography

[Fra+25] N. Frahm et al., “VIN-NBV: A View Introspection Network for Next-Best-View Selection.” [Online]. Available: https://arxiv.org/abs/2505.06219